

In class: 2/3

MT2c_VasconezJ

1 Midterm Exam (part 2) - Computational Physics I 6.5/10

1.0.1 Date: Thursday 3 April 2025

1.0.2 Credits: 10 points

1.0.3 Duration: 1 hour

1.1 Name: Juan Daniel Vasconez

1.1.1 Instructions:

- When you finish, please send me the **.ipynb** file via email to wbanda@yachaytech.edu.ec
- The exam is individual. Please include your name in the notebook.
- Within a **single python notebook**, solve the following problems:

1.2 5. I/O and regression: Larmor precession (5 points) 4/5

This problem consists of analysing the relationship between magnetic field strength and Larmor precession frequency, a fundamental principle in Nuclear Magnetic Resonance (NMR). To this end, a simulated dataset was generated, mimicking experimental measurements, and stored in a CSV file. This dataset contains pairs of values: **magnetic field strength (in Tesla)** and the corresponding observed **Larmor frequency (in Hertz)**. The goal is to investigate the relationship predicted by the **Larmor equation**.

Data: https://github.com/wbandabarragan/physics-teaching-data/blob/main/1D-data/larmor_precession_data.csv

- Create a pandas-based python I/O function that opens the above data file and returns 2 arrays, one for the magnetic field strength and one for the Larmor frequency.
- Call your I/O function and make a high-quality labeled plot of Larmor frequency (in the Y axis) versus magnetic field strength (in the X axis).
- Write down and use a physical model (motivated by quantum mechanics) to carry out a regression and constrain any free parameters in the model.
- Report the results from your regression, i.e. the best-fit value for the free parameter/s with respective uncertainty/ies.
- Make a plot showing the empirical data and the best-fit model obtained from your regression. Based on your analysis, can you infer the particle assumed for the simulated data?

Reference for Larmor precession: <https://github.com/wbandabarragan/quantum-mechanics-1/blob/main/unit-4/Lecture-Notes-UC4.pdf>

[26]: `import pandas as pd
import matplotlib.pyplot as plt
import scipy.optimize as opt
import numpy as np
import scipy.stats as st
from PIL import Image`

Larmor Precession is the precession of the magnetic moment of an object about an external magnetic field. Where the torque is defined as:

$$\vec{\tau} = \vec{\mu} \times \vec{B} = \gamma \vec{J} \times \vec{B}$$

where $\vec{\tau}$ is the torque, $\vec{\mu}$ is the magnetic dipole moment, $\vec{J}$ is the angular momentum vector, $\vec{B}$ is the external magnetic field.

The angular momentum vector $\vec{J}$ precesses about external fields with an angular frequency known as the Larmor Frequency defined as:

$$\omega = |\gamma B|$$

where ω is the angular frequency, B is the magnitude of the applied magnetic field, and γ is the gyromagnetic ratio. Being this our model to fit.

[3]: `def data(filename):
 """
 This function reads a file and returns its contents.

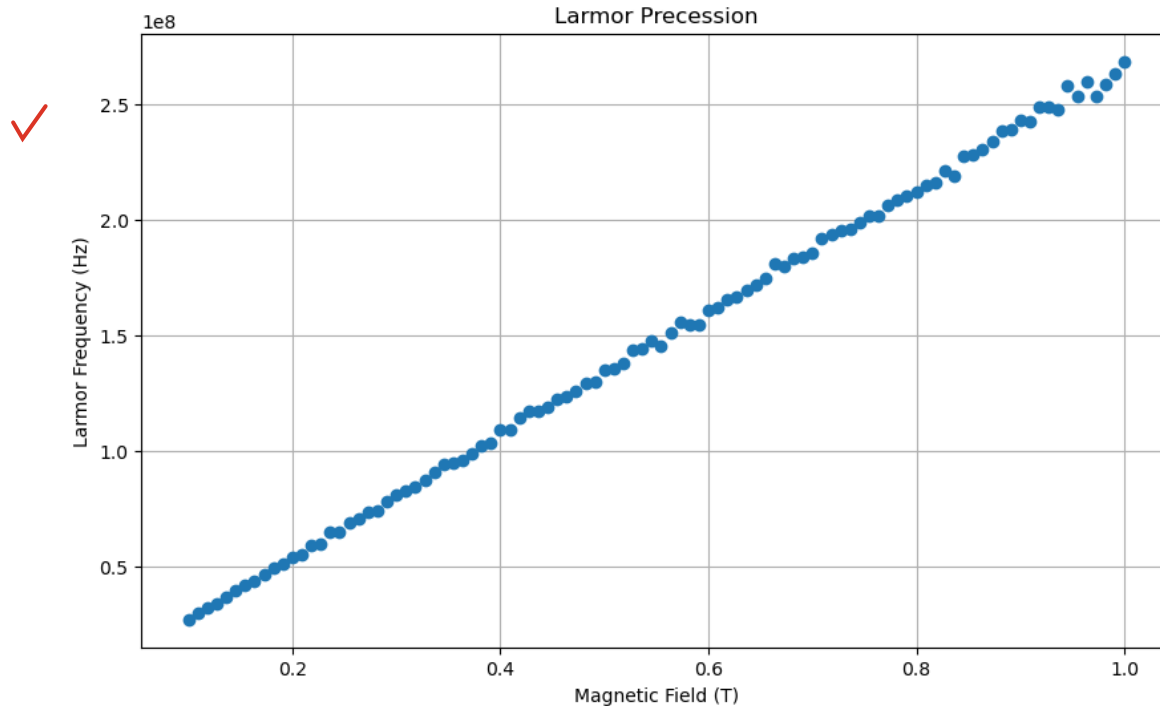
 :param filename: The name of the file to read
 :return: The contents of the file
 """

 df = pd.read_csv(filename)
 b = df['B (Tesla)'].to_numpy()
 lf = df['Frequency (Hz)'].to_numpy()

 return b, lf

b, lf = data('./larmor_precession_data.csv')

plt.figure(figsize=(10, 6))
plt.plot(b, lf, 'o', label='Data')
plt.xlabel('Magnetic Field (T)')
plt.ylabel('Larmor Frequency (Hz)')
plt.title('Larmor Precession')
plt.grid()
plt.show()`



Here we can denote the linear relation between our two variables. Making us able to do a good regression. Lets check our Pearson and Spearman Coefs

```
[4]: p = st.pearsonr(b, lf)
      print(f'Pearson correlation coefficient: {p[0]}')
      sp = st.spearmanr(b, lf)
      print(f'Spearman correlation coefficient: {sp[0]}')
```

Pearson correlation coefficient: 0.9997109287021749
Spearman correlation coefficient: 0.9997599759975996

Since we have a strong relation between our variables. lets define a model to fit. And fit the function based on the data using Curve Fit

```
[23]: def fit_model(x, m , b):
        """
        This function defines a linear model for fitting.

        :param x: The independent variable
        :param m: The slope of the line
        :param b: The y-intercept of the line
        :return: The dependent variable
        """
        y = m * x + b
        return y
```

```

coef, cova = opt.curve_fit(fit_model, b, lf)
gamma, b_fit = coef

err = np.sqrt(np.diag(cova))

print(err)
print('Fitted Gamma is:')
print("{:.7e}".format(gamma) + ' [Rad * Hz/T]')
print("Error in gamma is %.8e" % (err[0]) + ' [Rad * Hz/T]')

print("\n")
print('Fitted intercept is:')
print("{:.4e}".format(b_fit))
print("Error in intercetp is %.5f" % (err[1]))
print("\n")

print(f'Correlation coefficient: {np.corrcoef(b, lf)[0, 1]}')
print(f'The resulting Fit function is: Larmor Frequency = {gamma} * Magnetic_
↵Field + {b_fit}')

y = fit_model(b, gamma, b_fit)

```

[648047.88737561 394918.53694624]

Fitted Gamma is:

2.6675293e+08 [Rad * Hz/T] **✗ Units are off.**

Error in gamma is 6.48047887e+05 [Rad * Hz/T] **-0.5**

Fitted intercept is:

2.3013e+05

Error in intercetp is 394918.53695

Factor of 2*pi is missing

Correlation coefficient: 0.9997109287021748

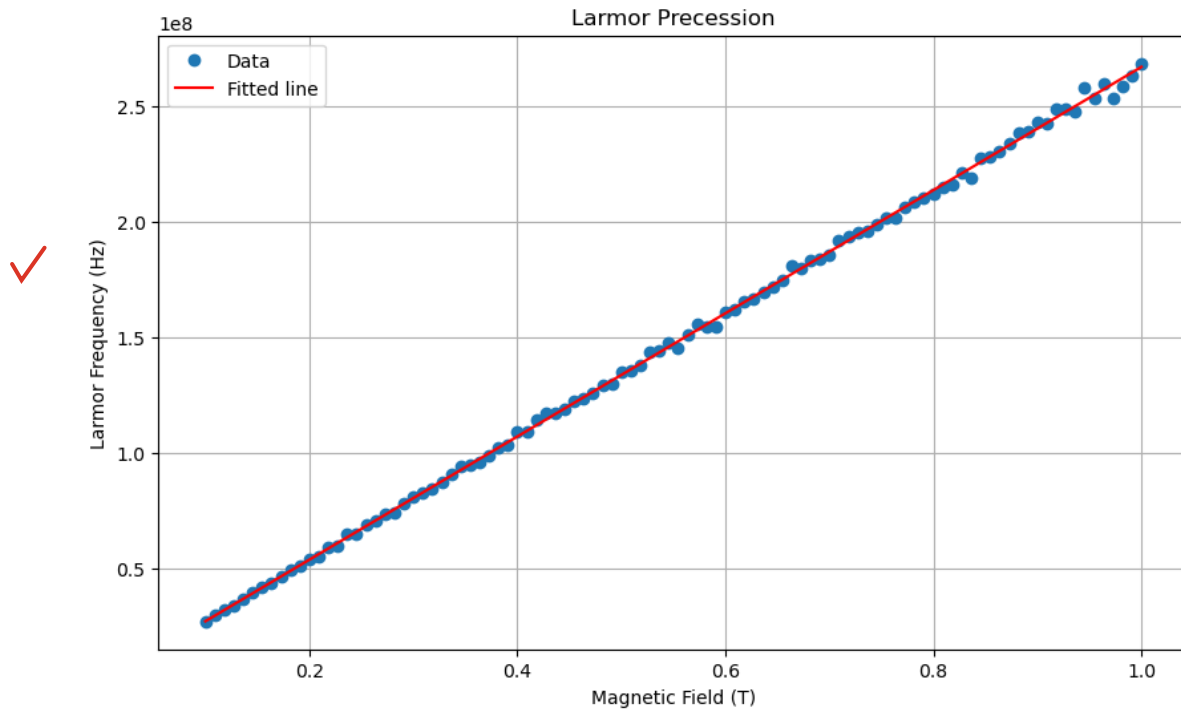
The resulting Fit function is: Larmor Frequency = 266752928.88981372 * Magnetic Field + 230129.4038747102

```

[20]: plt.figure(figsize=(10, 6))
plt.plot(b, lf, 'o', label='Data')
plt.plot(b, y, 'r-', label='Fitted line')
plt.xlabel('Magnetic Field (T)')
plt.ylabel('Larmor Frequency (Hz)')
plt.title('Larmor Precession')

```

```
plt.legend()  
plt.grid()  
plt.show()
```



Where, from this table found on wikipedia. Source: https://en.wikipedia.org/wiki/Gyromagnetic_ratio

Nucleus	γ_n ($10^6 \text{ rad}\cdot\text{s}^{-1}\cdot\text{T}^{-1}$)	$\gamma_n / (2\pi)$ ($\text{MHz}\cdot\text{T}^{-1}$)
^1H	267.522 187 08(11) ^[15]	42.577 478 461(18) ^[16]
^1H (in H_2O)	267.515 3194(11) ^[17]	42.576 385 43(17) ^[18]
^2H	41.065	6.536
^3H	285.3508	45.415 ^[19]
^3He	-203.789 460 78(18) ^[20]	-32.434 100 033(28) ^[21]
^7Li	103.962	16.546
^{13}C	67.2828	10.7084
^{14}N	19.331	3.077
^{15}N	-27.116	-4.316
^{17}O	-36.264	-5.772
^{19}F	251.815	40.078
^{23}Na	70.761	11.262
^{27}Al	69.763	11.103
^{29}Si	-53.190	-8.465
^{31}P	108.291	17.235
^{57}Fe	8.681	1.382
^{63}Cu	71.118	11.319
^{67}Zn	16.767	2.669
^{129}Xe	-73.995 401(2)	-11.776 7338(3) ^[22]

-0.5

We can infer that based on the gyromagnetic ratio. Our particle is Hydrogen



1.3 6. Image processing: Scanning Electron Microscope of Escherichia coli (5 points) 2.5/5

This problem consists of analysing the reduction of the **bacteria Escherichia coli (E. coli)** after treatment with **Zinc oxide (ZnO) nanoparticles**. The images provided below were obtained from a **Scanning Electron Microscope (SEM)**, before and after 5 hours of treatment with ZnO. One pixel of the image has an area of $0.02^2 \mu m^2$.

SEM image before ZnO treatment: https://github.com/wbandabarragan/physics-teaching-data/blob/main/2D-data/bacteria_before.jpg

SEM image after ZnO treatment: https://github.com/wbandabarragan/physics-teaching-data/blob/main/2D-data/bacteria_after.jpg

To analyse how effective this antibacterial treatment is, carry out the following analysis in python:

- a) Create a python function that reads in an image file and returns one its channels as an array.
- b) Call your function from (a), and:
 - Make a 2-panel figure showing the before and after SEM images.
 - Make a 2-panel figure showing the before and after pixel histograms of the SEM images.
- c) Create a python function that isolates the area covered by the E. coli bacteria from the background. Your function should read in an image channel array and return a cleaned binary image array with zeroes in the background and ones in the E. coli area.
- d) Call your function from (c), and:
 - Make a 2-panel figure showing the before and after binary images.
 - Make a 2-panel figure showing the before and after pixel histograms of the binary images.
- e) Using your binary images, calculate the following:
 - The total area in pixels and physical units covered by the E. coli bacteria before and after treatment.
 - The ratio between such areas, how effective is the antibacterial treatment with ZnO nanoparticles?

SEM images reference: Zhang et al. 2007, <https://link.springer.com/article/10.1007/s11051-006-9150-1>

```
[42]: def image_channels(filename):  
    """  
    This function reads an image and returns  
    the channels of the image.  
    """  
    img = plt.imread(filename)  
    return img  
  
after = image_channels('./bacteria_after.jpg')  
before = image_channels('./bacteria_before.jpg')  
print(after[0].size)
```

```

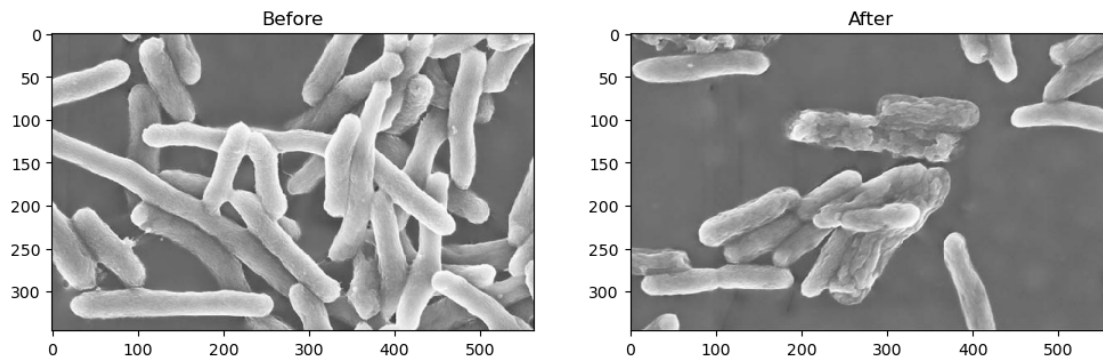
fig, ax = plt.subplots(1, 2, figsize=(12, 6))
ax[0].imshow(before)
ax[0].set_title('Before')
ax[1].imshow(after)
ax[1].set_title('After')
plt.show()

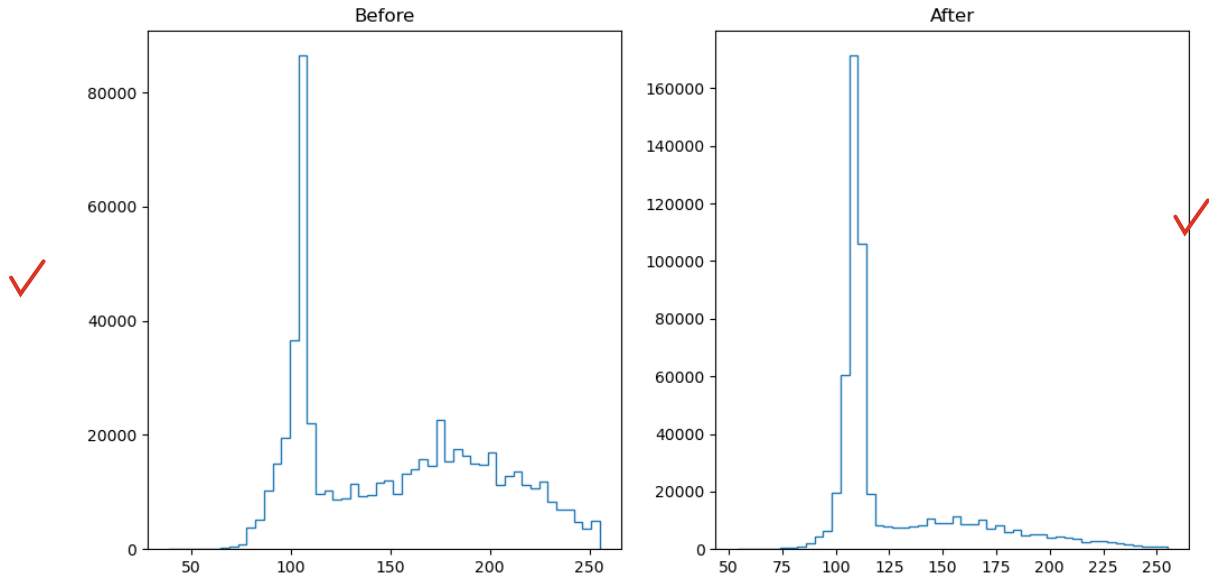
before1D = np.reshape(before, (before.size, 1))
after1D = np.reshape(after, (after.size, 1))

fig , ax = plt.subplots(1, 2, figsize=(12, 6))
ax[0].hist(before1D, histtype = "step", bins = 50)
ax[0].set_title('Before')
ax[1].hist(after1D, histtype = "step", bins = 50)
ax[1].set_title('After')
plt.show()

```

1689





```
[44]: def clean_image_data(before, after, threshold1=10000, threshold2=20000):
    """
    This function reshapes the before and after images into 1D arrays
    and applies a threshold to create binary cleaned arrays.

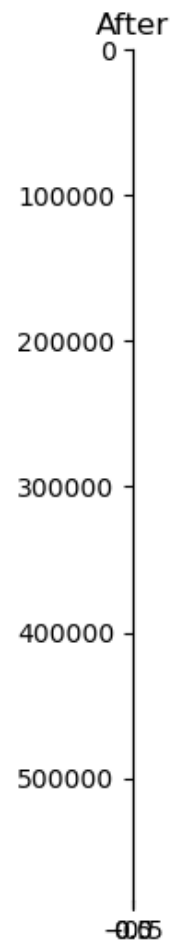
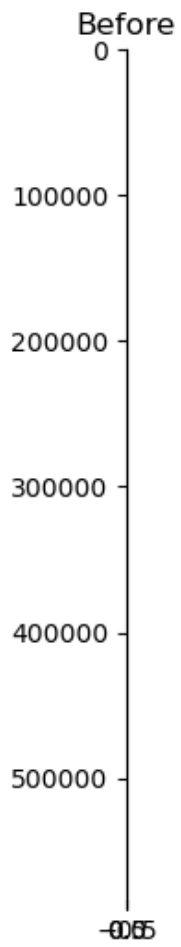
    :param before: numpy.ndarray, the 'before' image data
    :param after: numpy.ndarray, the 'after' image data
    :param threshold: int, the threshold value for cleaning
    :return: tuple of numpy.ndarrays, cleaned 1D arrays for before and after_
    ↪ images
    """
    after1D = np.reshape(after, (after.size, 1))
    before1D = np.reshape(before, (before.size, 1))
    after1D_clean = np.where(after1D < threshold1, 0., 1.)
    before1D_clean = np.where(before1D < threshold2, 0., 1.)
    return before1D_clean, after1D_clean

before1D_clean, after1D_clean = clean_image_data(before, after)

fig, ax = plt.subplots(1, 2, figsize=(12, 6))
ax[0].imshow(before1D_clean, cmap='gray')
ax[0].set_title('Before')
ax[1].imshow(after1D_clean, cmap='gray')
ax[1].set_title('After')
plt.show()
```

arrays are flattened.

✗ -0.5



[]:

d,e are missing

-2.0